Algebra of Programming

Bird & de Moor, chapters 5–10 — in Freyd diagram order

Composition is juxtaposition in diagram order: RS is "first R, then S", so every composite here is the reverse of the book's. A row tagged ✓ has its mirrored statement machine-checked under `AOP/A*.lean`.

5 Datatypes in Allegories

5.1 Relators

5.1 ✓ **RELATOR** $F : \mathcal{A} \to \mathcal{B}$ between tabular allegories $\triangleq$ a *monotonic* functor: $R \subset S \Longrightarrow FR \subset FS$.

Lem 5.1 ✓ *F a relator and f a map $\Longrightarrow$ Ff is a map and* $(Ff)^\circ = F(f^\circ)$.

Thm 5.1 ✓ *A functor is a relator $\iff$ it preserves converse:* $F(R^\circ) = (FR)^\circ$.

Cor 5.1 ✓ *Relators agreeing on maps are equal:* $(\forall f.\, Ff = Gf) \Longrightarrow F = G$. Tabulate: $FR = F(f^\circ g) = (Ff)^\circ(Fg)$.

Hence FR° is unambiguous — it names $(FR)^\circ$ and $F(R^\circ)$ alike.

Ex 5.2 ✓ A relator preserves meets of coreflexives, $F(C \cap D) = FC \cap FD$. The restriction $C, D \subset 1$ is needed.

Ex 5.5 ✓ $F(\mathrm{dom}\ R) = \mathrm{dom}(FR)$.

Ex 5.4 Not every functor on Fun extends to a relator: for $FA = \emptyset$ if $A = \emptyset$ and $\{0\}$ otherwise, the constant maps $\mathrm{one}, \mathrm{two} : \{0\} \to \{1,2\}$ give $F(\mathrm{two}\,\mathrm{one}^\circ) = 1$, while preservation of converse forces $\emptyset$.

5.2 Relational products

(π, π') tabulates Π, so $\langle f, g\rangle = (f\pi^\circ) \cap (g\pi'^\circ)$ for maps; (5.1) reads that same formula for arbitrary relations.

(5.1) ✓ $\langle R, S\rangle \triangleq (R\pi^\circ) \cap (S\pi'^\circ)$. Rel $c\,\langle R, S\rangle\,(a, b) \iff cRa \wedge cSb$.

(5.2) ✓ $R \times S \triangleq \langle \pi R, \pi' S\rangle$.

$\times$ is monotonic in both arguments and $(R \times S)^\circ = R^\circ \times S^\circ$, so it is a relator — but it is *not* a categorical product.

(5.3) ✓ **ABSORPTION** $\langle X, Y\rangle(R \times S) = \langle XR, YS\rangle$; hence $(R \times S)(U \times V) = (RU) \times (SV)$.

(5.4) ✓ $\langle XR, Y\rangle \subset \langle X, Y\rangle(R \times 1)$ — one of the two special cases (5.3) is staged through.

(5.5) ✓ $\langle X, YS\rangle \subset \langle X, Y\rangle(1 \times S)$, symmetric to (5.4).

(5.6) ✓ $\langle R, S\rangle\pi = (\mathrm{dom}\ S)R$.

(5.7) ✓ $\langle R, S\rangle\pi' = (\mathrm{dom}\ R)S$.

So $\langle R, \emptyset\rangle\pi = \emptyset$, not R: precisely why (π, π') is no categorical product in an allegory.

(5.8) ✓ **CANCELLATION** $\langle X, Y\rangle\langle R, S\rangle^\circ = (XR^\circ) \cap (YS^\circ)$.

Ex 5.6 ✓ $(1 \times S)(R \times 1) \supset R \times S$, from (5.4) and (5.2) alone.

Ex 5.7 $\langle R, S\rangle^\circ\langle P, Q\rangle \subset (R^\circ P) \times (S^\circ Q)$.

Ex 5.8 $\langle X, Y\rangle(R \times S) \subset \langle XR, YS\rangle$ — the easy half of (5.3).

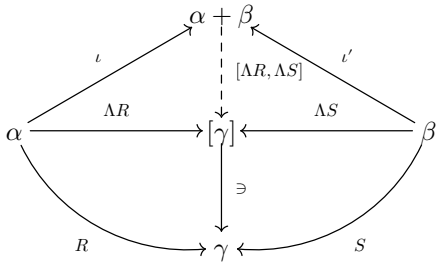
Ex 5.9 ✓ $f\langle R, S\rangle = \langle fR, fS\rangle$ for a map f; false for an arbitrary arrow.

Ex 5.10 $\mathrm{unzip}(F) \triangleq \langle F\pi, F\pi'\rangle$, and $F(R \times S)\,\mathrm{unzip}(F) = \mathrm{unzip}(F)\,(FR \times FS)$.

5.3 Relational coproducts

A coproduct of α, β in $\mathsf{Fun}(\mathcal{A})$ is a coproduct in the whole power allegory $\mathcal{A}$:

$$\iota T = R \wedge \iota' T = S \iff T = [\Lambda R, \Lambda S]\ni.$$



5.3 ✓ **JUNC** $[R, S] \triangleq [\Lambda R, \Lambda S]\ni$.

(5.9) ✓ $[R, S] = (\iota^\circ R) \cup (\iota'^\circ S)$.

(5.10) ✓ $R + S \triangleq [R\iota, S\iota']$, monotonic in both arguments, so $+$ is a relator.

(5.11) ✓ **CANCELLATION** $[U, V]^\circ[R, S] = (U^\circ R) \cup (V^\circ S)$. Easier than (5.8) — but still not the dual of it, an allegory being its own opposite.

Ex 5.12 ✓ $\iota[1, \emptyset] = 1$ and $\iota'[1, \emptyset] = \emptyset$, and $[1, \emptyset]\iota \cup [\emptyset, 1]\iota' = [\iota, \iota'] = 1$; hence $[1, \emptyset] = \iota^\circ$, $[\emptyset, 1] = \iota'^\circ$, which gives (5.9).

Ex 5.14 $(R + S) \cap ([P, Q][U, V]^\circ) = (R \cap PU^\circ) + (S \cap QV^\circ)$.

5.4 The power relator

5.4 ✓ On maps $Pf = ((\ni f)/\ni) \cap (\ni^\circ \setminus (f\ni^\circ))$; read as a definition for relations,

$$PR \triangleq ((\ni R)/\ni) \cap (\ni^\circ \setminus (R\ni^\circ)).$$

Rel $x\,(PR)\,y \iff (\forall a \in x.\, \exists b \in y.\, aRb) \wedge (\forall b \in y.\, \exists a \in x.\, aRb)$. Every element of x has an R-partner in y, and conversely.

✓ P is monotone in R, straight from monotonicity of division.

✓ $P1 = 1$ — this is the antisymmetry of subset $= \ni/\ni$ restated, so it needs the allegory unitary and tabular.

✓ $(PR)(PS) = P(RS)$, so P is a relator. $\subset$ is division cancellation; $\supset$ needs a tabulation (x, z) of $P(RS)$ and the mediating map $y = \Lambda((x\ni R^\circ) \cap (z\ni S))$.

✓ P and E agree on maps: $Pf = \Lambda(\ni f) = Ef$.

Ex 5.15 They differ on relations, and Corollary 5.1 does not object: E is not monotonic — distinct maps are never comparable, so $R \subset S$ says nothing about $\Lambda(\ni R)$ and $\Lambda(\ni S)$ — hence not a relator.

Ex 5.16 $P(\mathrm{dom}\ R)(ER) \subset PR$.

5.5 Relational catamorphisms

Let F have initial algebra $\mathsf{in} : FT \to T$ *in the subcategory of maps*. It is then initial for relational algebras as well.

(5.12) ✓ $\mathsf{in}\ X = (FX)R \iff X = (\!|\Lambda((F\ni)R)|\!)\ni$.

5.5 ✓ $(\!|R|\!) \triangleq (\!|\Lambda((F\ni)R)|\!)\ni$ — the unique X with $\mathsf{in}\ X = (FX)R$.

5.5 ✓ **EILENBERG–WRIGHT** $\Lambda(\!|R|\!) = (\!|\Lambda((F\ni)R)|\!)$: the transpose of a relational catamorphism is a catamorphism of maps. This is what turns a

relational specification into a functional fold over sets, and it is used throughout §5.6.

✓ Over a map algebra the relational catamorphism is the ordinary one.

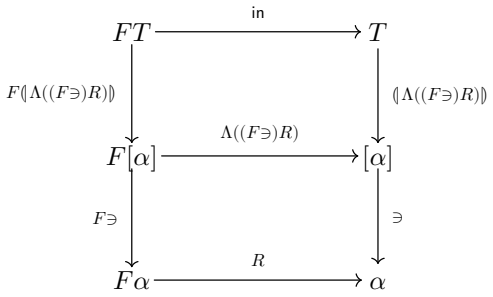
5.5 **TYPE RELATOR** F binary with initial type (α, T): then $(TR)^\circ = T(R^\circ)$, so T is a relator. By uniqueness — both sides solve in $X = F(R^\circ, X)$ in.

Ex 5.17 ✓ In a Boolean allegory each coreflexive C has a $\sim C$ with $C \cap \sim C = \emptyset$ and $C \cup \sim C = 1$. Then guard $C \triangleq [C, \sim C]^\circ$ is a map, $(C \to R, S) \triangleq (\text{guard } C)(R + S)$, and conditionals behave like cases:

$$R \subset (C \to S, T) \iff CR \subset S \wedge (\sim C)R \subset T.$$

Ex 5.18 F preserves meets $\Longrightarrow$ so does the type relator T it induces; in particular list does.

Ex 5.19 R entire $\Longrightarrow (\!|R|\!)$ entire (reflection gives $\text{dom}\,(\!|R|\!) = 1$).



5.6 Combinatorial functions

Lists mean cons-lists, and list is their type functor, list $R = (\!|\text{nil}, (R \times 1)\ \text{cons}|\!)$. Each relation below is specified as a catamorphism, transposed by Eilenberg–Wright, then implemented on lists.

Subsequences

5.6 subseq $= (\!|[\text{nil}, \text{cons} \cup \pi']|\!) : \text{list}\ \alpha \to \text{list}\ \alpha$ — at each step keep the head or drop it.

Λ subseq $= (\!|\text{nil}\ \tau, \langle \Lambda(1 \times \ni)(P\ \text{cons}), \pi' \rangle\ \text{cup}|\!)$, by expanding $\Lambda([\text{nil}, \text{cons} \cup \pi']\, F(1, \ni))$ for $F(A, B) = 1 + (A \times B)$. Pointwise: $e = \{[]\}$ and $f(a, xs) = \{\text{cons}(a, x) \mid x \in xs\} \cup xs$.

$\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle$ cup, where cup unions a pair of sets.

5.6 Sets replaced by lists: subseqs $= (\!|\text{nil wrap}, \langle \text{cpr (list cons)}, \pi' \rangle\ \text{cat}|\!)$, i.e. $e = [[]]$ and $f(a, xs) = [\text{cons}(a, x) \mid x \leftarrow xs] +\!\!+ xs$.

cpr $: \alpha \times \text{list}\ \beta \to \text{list}(\alpha \times \beta)$ ("cartesian product, right"), $\text{cpr}(a, x) = [(a, b) \mid b \leftarrow x]$.

5.6 setify $\triangleq (\!|\omega, (\tau \times 1)\ \text{cup}|\!) : \text{list}\ \alpha \to [\alpha]$, a list's set of elements; ω returns the empty set.

Ex 5.21 nil setify $= \omega$, wrap setify $= \tau$, cat setify $= (\text{setify} \times \text{setify})\text{cup}$, $(\text{list}\ f)$ setify $= \text{setify}\ (Pf)$.

concat setify $= (\text{list setify})\,\text{setify} \bigcup$, cpr setify $= (1 \times \text{setify})\Lambda(1 \times \ni)$.

Ex 5.22 subseqs setify $= \Lambda$ subseq — the list program implements the set specification. By fusion, from the identities above.

Ex 5.23 subseq is the converse of a catamorphism, the one computing supersequences.

Cartesian product

5.6 $\text{cpp}(x, y) = [(a, b) \mid a \leftarrow x, b \leftarrow y]$ and $\text{cpl}(x, b) = [(a, b) \mid a \leftarrow x]$, with cpp setify $= \Lambda(\ni \times \ni)$ and cpl setify $= \Lambda(\ni \times 1)$.

5.6 $\text{cp}(F) \triangleq \Lambda(F\ni) : F[\alpha] \to [F\alpha]$ — one choice from each slot. cpr, cpp, cpl implement it for $FA = \alpha \times A$, $A \times A$ and $A \times \beta$.

$\text{cp(list)} = (\!|\Lambda[\text{nil}, (\ni \times \ni)\text{cons}]|\!) = (\!|\text{nil}\ \tau, \Lambda(\ni \times \ni)(P\ \text{cons})|\!)$ by Eilenberg–Wright; $\text{cp(list)}[x_1, ..., x_n] = \{[a_1, ..., a_n] \mid a_j \in x_j\}$. On lists: cplist $= (\!|\text{nil wrap}, \text{cpp (list cons)}|\!)$.

Ex 5.20 ✓ $\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle$ cup, $\Lambda(R \cap S) = \langle \Lambda R, \Lambda S \rangle$ cap, $\Lambda(R \times S) = (\Lambda R \times \Lambda S)$ cross, with cup $= \Lambda((\pi\ni) \cup (\pi'\ni))$ and cap $= \Lambda((\pi\ni) \cap (\pi'\ni))$.

Prefix and suffix

5.6 prefix $= \text{cat}^\circ\pi$: x prefix y when $x +\!\!+ z = y$ for some z. Also prefix $= \text{init}^*$ with init $= \text{snoc}^\circ\pi$, and prefix $= (\!|\text{nil}, \text{nil} \cup \text{cons}|\!)$ — the second nil being $!\,\text{nil} : \alpha \times \text{list}\ \alpha \to \text{list}\ \alpha$.

Λ prefix $= (\!|\text{nil}\ \tau, \langle \text{nil}\ \tau, \Lambda(1 \times \ni)(P\ \text{cons}) \rangle\ \text{cup}|\!)$.

5.6 inits $= (\!|\text{nil wrap}, \langle \text{nil wrap}, \text{cpr (list cons)} \rangle\ \text{cat}|\!)$, i.e. inits $[] = [[]]$ and inits $([a] +\!\!+ x) = [[]] +\!\!+ [[a] +\!\!+ y \mid y \leftarrow \text{inits}\ x]$ — initial segments in increasing order of length.

5.6 suffix $= \text{tail}^*$ with tail $= \text{cons}^\circ\pi'$, the snoc-list dual of prefix. tails $[] = [[]]$ and tails $(x +\!\!+ [a]) = [y +\!\!+ [a] \mid y \leftarrow \text{tails}\ x] +\!\!+ [[]]$ — decreasing length.

Not implementable as it stands; as a cons-list catamorphism tails $= (\!|\text{nil wrap}, \text{extend}|\!)$ with $\text{extend}(a, [x] +\!\!+ xs) = [[a] +\!\!+ x] +\!\!+ [x] +\!\!+ xs$.

5.6 splits $= \langle \text{inits}, \text{tails} \rangle$ zip implements $\Lambda(\text{cat}^\circ)$, every way of splitting a list in two — and works only because inits and tails order their segments oppositely.

Ex 5.24 init $: \text{list}^+\alpha \to \text{list}\ \alpha$ is itself a catamorphism.

Partitions

5.6 partition $\triangleq \text{concat}^\circ : \text{list}\ \alpha \to \text{list}(\text{list}^+\alpha)$ — cut a list into non-empty contiguous segments. Rel partition $[1, 2, 3] = \{[[1], [2], [3]], [[1, 2], [3]], [[1], [2, 3]], [[1, 2, 3]]\}$.

As a catamorphism partition $= (\!|\text{nil}, \text{new} \cup \text{glue}|\!)$ with new $= (\text{wrap} \times 1)\text{cons}$ and glue $= (1 \times \text{cons}^\circ)\ \text{assocl}\ (\text{cons} \times 1)\ \text{cons}$: at each step either open a segment or glue the element onto the first one. Pointwise $\text{new}(a, xs) = [[a]] +\!\!+ xs$ and $\text{glue}(a, [x] +\!\!+ xs) = [[a] +\!\!+ x] +\!\!+ xs$.

6 The two agree by the converse-catamorphism theorem of the next chapter: ran $R = 1$ and $Rf \subset F(1, f)$ in $\Longrightarrow f^\circ = (\!|R|\!)$, applied to $f = \text{concat}$ and $R = [\text{nil}, \text{new} \cup \text{glue}]$.

Ex 5.25 Its surjectivity rests on new$^\circ$ new $=$ cons$^\circ$(wrap$^\circ$ wrap $\times$ 1) cons and glue$^\circ$ glue $=$ cons$^\circ$(cons$^\circ$ cons $\times$ 1) cons.

Λ partition $= (\!|\Lambda\ \text{nil}, \Lambda((1 \times \ni)(\text{new} \cup \text{glue}))|\!)$, whose step simplifies (Ex 5.27) to $\Lambda(1 \times \ni)\ P(\langle \text{new}\ \tau, \Lambda\ \text{glue} \rangle\ \text{cup})\bigcup$.

5.6 partitions $=$ $(\!|\text{nil wrap}, \text{cpr (list } (\langle \text{new}, \text{glues} \rangle\ \text{cons})) \text{ concat}|\!)$, where glues implements Λ glue: $\text{glues}(a, []) = []$ and $\text{glues}(a, [x] +\!\!+ xs) = [[[a] +\!\!+ x] +\!\!+ xs]$.

Ex 5.26 partitions setify $= \Lambda$ partition.

Permutations

5.6 bag α is the initial algebra [bnil, bcons] of $F(A, B) = 1 + A \times B$ whose bcons commutes, $(1 \times \text{bcons})\text{bcons} = \text{exch}\ (1 \times \text{bcons})\ \text{bcons}$, i.e. $\text{bcons}(a, \text{bcons}(b, x)) = \text{bcons}(b, \text{bcons}(a, x))$: order is

irrelevant, duplicates are not. exch = assocl (swap × 1) assocr.

5.6 bagify = (|bnil, bcons|) : list $\alpha \to$ bag α is surjective, bagify° bagify = 1.

5.6 perm $\triangleq$ bagify bagify° — equal bags of values. At once perm = perm°.

As a catamorphism perm = (|nil, add|) with add = cons perm, or with the usable add = $(1 \times \text{cat}^\circ)$ exch $(1 \times \text{cons})$ cat: $\text{add}(a, x) = y +\!\!+ [a] +\!\!+ z$ where $y +\!\!+ z = x$, so a goes in anywhere.

Ex 5.28 cons perm = $(1 \times \text{perm})$ cons perm, from bagify° bagify = 1.

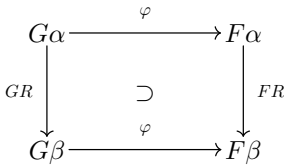
5.6 adds$(a, x) = [y +\!\!+ [a] +\!\!+ z \mid (y, z) \leftarrow \text{splits } x]$ and perms = (|nil wrap, cpr (list add)|).

Ex 5.29 interleave, all interleavings of two lists, is the converse of a catamorphism.

5.7 Lax natural transformations

Some equalities of the functional world become inequalities in the relational one.

(5.13) ✓ LAX NATURAL $\varphi : F \hookrightarrow G \triangleq \varphi(FR) \supset (GR)\varphi$ for every R, where $\varphi_\alpha : G\alpha \to F\alpha$. The $\supset$ is the way the hook points.



5.7 Example: $\ni : 1 \hookrightarrow P$, that is $(PR)\ni \subset \ni R$ — immediate from the definition of PR.

Thm 5.2 ✓ *F, G relators: φ is lax natural $F \hookrightarrow G \iff \varphi$ is strictly natural on maps, $\varphi(Ff) = (Gf)\varphi$.* So lax naturality adds nothing to naturality of the underlying map functors — it only records how it extends.

Ex 5.30 Three instances: $R\tau \subset \tau(PR)$, $R\langle 1, 1\rangle \subset \langle 1, 1\rangle(R \times R)$, $(PR \times PR)$ cup $\subset$ cup (PR).

6 Recursive Programs

6.1 Digits of a number

Convert a positive natural to its decimal digits. The pattern of the whole chapter in one example: specify a function as a refinement of some relation, discover that the relation solves a recursion equation, then read the recursion off as a program.

6.1 Decimal ::= wrap Digit^+ | snoc(Decimal, Digit), so ([wrap, snoc], Decimal) is the initial algebra of $FA = \text{Digit}^+ + (A \times \text{Digit})$.

val = (|embed, op|) : Decimal $\to \text{Nat}^+$, with embed the inclusion of digits and $\text{op}(n, d) = 10n + d$.

(6.1) digits $\subset$ val° — a *functional refinement* of val°, which is not yet known to be a map. This is the specification.

val°
= definition
(|embed, op|)°
= catamorphisms
([wrap, snoc]°(F val)[embed, op])°
= converse
[embed, op]°(F val°)[wrap, snoc]
= definition of F
[embed, op]°(1 + (val° × 1))[wrap, snoc]
= coproduct
[embed, op]°[wrap, (val° × 1) snoc]
= coproduct
(embed° wrap) ∪ (op°(val° × 1) snoc)

6.1 ✓ So val° = (embed° wrap) ∪ (op°(val° × 1) snoc).

op$(n, d) = m \iff n = m \text{ div } 10 \wedge d = m \bmod 10$, so op° is defined exactly for $m \geq 10$ and embed° exactly for $m < 10$: the two branches have disjoint domains, and the join becomes a conditional.

val°m = wrap m if $m < 10$, else snoc(val°$(m \text{ div } 10), m \bmod 10$). The recursion terminates because $m \geq 10 \implies m \text{ div } 10 < m$, so val° is a total function and digits = val°.

digits $m = [m]$ if $m < 10$, else digits$(m \text{ div } 10) +\!\!+ [m \bmod 10]$ — quadratic, because snoc on lists is linear.

With an accumulator, digits $m = f(m, [])$ where $f(m, x) = [m] +\!\!+ x$ if $m < 10$, else $f(m \text{ div } 10, [m \bmod 10] +\!\!+ x)$ — linear, and correct at 0.

6.2 Least fixed points

T 6.1 ✓ KNASTER–TARSKI *φ a monotonic mapping on the arrows $\alpha \to \beta$ of a locally complete allegory. Then $\varphi X \subset X$ and $\varphi X = X$ have the same least solution, written $(\mu X : \varphi X)$; dually $X \subset \varphi X$ and $X = \varphi X$ have the same greatest solution.*

Proof: $(\mu X : \varphi X) = \bigcap\{X : \varphi X \subset X\}$. Local completeness is what makes that meet exist.

in is an isomorphism, so (|R|) is the unique — hence both least and greatest — solution of $X = \text{in}^\circ(FX)R$, and $\varphi X = \text{in}^\circ(FX)R$ is monotonic. Knaster–Tarski then gives:

(6.2) ✓ (|R|) $\subset X \Longleftarrow \text{in}^\circ(FX)R \subset X$.

(6.3) ✓ $X \subset$ (|R|) $\Longleftarrow X \subset \text{in}^\circ(FX)R$.

(6.4) ✓ FUSION, $\subset$ (|T|) $\subset$ (|R|)$S \Longleftarrow (FS)T \subset RS$.

(6.5) ✓ **FUSION, ⊃** $(\!|R|\!)S \subset (\!|T|\!) \Longleftarrow RS \subset (FS)T$. Equality of the two hypotheses gives back the catamorphism fusion law of ch. 3.

Ex 6.3 ✓ **KLEENE** φ *continuous* — it preserves joins of ascending chains — then $(\mu X : \varphi X) = \bigcup\{\varphi^n \emptyset : 0 \leq n\}$.

Ex 6.4 ✓ **FIXED POINT INDUCTION** $(\mu X : \varphi X) \subset A \Longleftarrow \varphi A \subset A$; by Kleene, $(\forall X.\, X \subset A \Longrightarrow \varphi X \subset A)$ suffices too.

Ex 6.5 The least solution of $\varphi X \subset X$ satisfies $\varphi X = X$ — Lambek's lemma, for the containment order read as a category and φ as a functor on it.

Ex 6.7 ✓ $(\!|R|\!) \subset (\!|S|\!) \Longleftarrow (F(\!|S|\!))R \subset (F(\!|S|\!))S$. It does *not* need $R \subset S$.

Ex 6.8 ✓ R **DIFUNCTIONAL** $\triangleq R = RR^\circ R$; the difunctional closure of R is a least fixed point.

6.3 Hylomorphisms

6.3 A **HYLOMORPHISM** is a catamorphism after the converse of one, $(\!|S|\!)^\circ(\!|R|\!)$ for $R : F\alpha \to \alpha$ and $S : F\beta \to \beta$. The initial type μF is the *intermediate data structure*: it is built by $(\!|S|\!)^\circ$ and consumed by $(\!|R|\!)$, and never appears in the answer.

T 6.2 ✓ **HYLOMORPHISM THEOREM**

$$(\!|S|\!)^\circ(\!|R|\!) = (\mu X : S^\circ(FX)R).$$

S° divides, FX conquers, R combines.

Proof in two halves: $(\!|S|\!)^\circ(\!|R|\!)$ is a fixed point (functors and the two catamorphism laws), and it is below every prefixed point — that half goes through division, $(\!|S|\!)^\circ(\!|R|\!) \subset X \Longleftrightarrow (\!|R|\!) \subset X/(\!|S|\!)^\circ$, which is the standard move for least fixed points.

C 6.1 ✓ $FX = GX + HX$, so an F-algebra is a pair: $(\!|S_1, S_2|\!)^\circ(\!|R_1, R_2|\!) = (\mu X : (S_1^\circ(GX)R_1) \cup (S_2^\circ(HX)R_2))$.

in is an isomorphism and $(\!|\mathsf{in}|\!) = 1$, so every catamorphism and every converse of one is already a hylomorphism.

Ex 6.10 ✓ Hylomorphisms preserve simplicity: R and S simple $\Longrightarrow (\!|S|\!)^\circ(\!|R|\!)$ simple.

6.4 Fast exponentiation and modulus

6.4 $\exp\, a = (\!|\mathrm{one}, \mathrm{mult}\ a|\!) : \mathrm{Nat} \to \mathrm{Nat}$ encodes $a^0 = 1$ and $a^{b+1} = a \times a^b$ — $O(b)$ steps. Divide and conquer brings it to $O(\log b)$.

Bin = listl Bit, Bit $= \{0, 1\}$; convert $= (\!|\mathrm{zero}, \mathrm{shift}|\!) :$ Bin $\to$ Nat with $\mathrm{shift}(n, d) = 2n + d$ reads a bit sequence as a number. It is simple.

$$\begin{array}{ll} & \exp\, a \\ \supseteq & \text{convert simple, so convert}^\circ\ \text{convert} \subset 1 \\ & \mathrm{convert}^\circ\ \mathrm{convert}\ (\exp\, a) \\ = & \text{fusion, with op } a(n,d) = (d = 0 \to n^2, a \times n^2) \\ & \mathrm{convert}^\circ (\!|\mathrm{one}, \mathrm{op}\ a|\!) \\ = & \text{Corollary 6.1} \\ & (\mu X : (\mathrm{zero}^\circ\ \mathrm{one}) \cup (\mathrm{shift}^\circ (X \times 1)(\mathrm{op}\ a))) \end{array}$$

6.4 ✓ The fusion step needs $\mathrm{zero}\ (\exp\, a) = \mathrm{one}$ and $\mathrm{shift}\ (\exp\, a) = (\exp\, a \times 1)(\mathrm{op}\ a)$. zero and shift have disjoint ranges, so the join is again a conditional:

$\exp\, a\, b = 1$ if $b = 0$, else $\mathrm{op}\ a(\exp\, a(b \text{ div } 2), b \bmod 2)$.

6.4 ✓ Same idea for $a \bmod b$: $\mathrm{mod}\ b = (\!|\mathrm{zero}, \mathrm{succ}\ b|\!)$ with $\mathrm{succ}\ b\, a = (a = b - 1 \to 0, a + 1)$ costs $O(a)$; fusing through convert gives

$$\mathrm{mod}\ b \supset (\mu X : (\mathrm{zero}^\circ\ \mathrm{zero}) \cup (\mathrm{shift}^\circ (X \times 1)(\mathrm{op}\ b)))$$

with $\mathrm{op}\ b(r, d) = (n \geq b \to n - b, n)$ and $n = 2r + d$.

Conditions: $\mathrm{zero}\ (\mathrm{mod}\ b) = \mathrm{zero}$ and $\mathrm{shift}\ (\mathrm{mod}\ b) = (\mathrm{mod}\ b \times 1)(\mathrm{op}\ b)$. The program is $\mathrm{mod}\ b\, a = 0$ if $a = 0$; $\mathrm{op}\ b(\mathrm{mod}\ b(a \text{ div } 2), 0)$ if a even; $\mathrm{op}\ b(\mathrm{mod}\ b(a \text{ div } 2), 1)$ if a odd — $O(\log a)$.

6.5 Unique fixed points

A hylomorphism is the *least* fixed point of its recursion, not necessarily the only one.

6.5 On Nat take $X = (\!|\mathrm{zero}, \mathrm{positive}|\!)^\circ(\!|\mathrm{zero}, 1|\!)$ with positive $=$ succ$^\circ$ succ the coreflexive on positives. Both catamorphisms describe the coreflexive on 0, so X is that. But its recursion $X = (\mathrm{zero}^\circ\ \mathrm{zero}) \cup (\mathrm{positive}\ X)$ also has $X = 1$ as a solution.

$[\mathrm{zero}, \mathrm{positive}]^\circ$ and $[\mathrm{zero}, 1]$ are both maps, yet the least solution is not even entire. Simplicity does carry over (Ex 6.10); entireness is what needs a hypothesis, and the hypothesis is that a membership relation be *inductive*.

Inductive and well-founded relations

6.5 ✓ $R : \alpha \to \alpha$ is **INDUCTIVE** $\triangleq$ for all X,

$$X/R \subset X \Longrightarrow \Pi \subset X.$$

Pointwise: if $(\forall c.\, cRa \Longrightarrow cXb) \Longrightarrow aXb$ holds for all a, b, then X holds everywhere — exactly a proof by induction over R.

$<$ on Nat gives ordinary mathematical induction; $\mathrm{tail} = \mathrm{cons}^\circ \pi'$ gives induction over lists.

Ex 6.13 ✓ S inductive and $RR \subset SR \Longrightarrow R$ inductive. Hence $R \subset S$ with S inductive $\Longrightarrow R$ inductive, and S inductive $\Longleftrightarrow S^+$ inductive, where $S^+ = (\mu X : S \cup (SX))$.

Ex 6.15 ✓ The meet of two inductive relations is inductive; the join need not be.

6.5 ✓ R is **WELL-FOUNDED** $\triangleq$ for all X, $X \subset RX \Longrightarrow X = \emptyset$ — no infinite chain $a_0 R a_1 R \ldots$ Inductive $\Longrightarrow$ well-founded, and the converse holds in a Boolean allegory.

Ex 6.16 ✓ R well-founded $\Longrightarrow fRf^\circ$ well-founded, for any map f.

Membership

6.5 A relator F may carry a **MEMBERSHIP** arrow $\mathrm{mem}(F)_\alpha : F\alpha \to \alpha$ with $a\ \mathrm{mem}\ x$ exactly when a is an element of x. For the polynomial relators, the power relator and the type relators it exists:

$\mathrm{mem}(\mathrm{Id}) = 1$, $\mathrm{mem}(K_\beta) = \emptyset$, $\mathrm{mem}(F + G) = [\mathrm{mem}(F), \mathrm{mem}(G)]$.

$\mathrm{mem}(F \times G) = (\pi\, \mathrm{mem}(F)) \cup (\pi'\, \mathrm{mem}(G))$, $\mathrm{mem}(F \circ G) = \mathrm{mem}(F)\, \mathrm{mem}(G)$, $\mathrm{mem}([\cdot]) = \ni$.

Type relators, via sets: $\mathrm{mem}(T) = \mathrm{setify}(T) \ni$ with $\mathrm{setify}(T) = \Lambda(\mathrm{mem}(T))$ — Ex 6.17 gives a set-free alternative.

mem is a lax natural transformation $1 \hookrightarrow F$, that is $(FR)\ \mathrm{mem} \subset \mathrm{mem}\ R$, and it is the *largest* one of that type — so a membership relation, if it exists, is unique.

6.5 ✓ The result the section is for: $\mathsf{in}^\circ\ \mathrm{mem}(F)$ is inductive. It is the immediate-subterm relation.

Nat, $FX = 1 + X$: mem $= [\emptyset, 1]$ and $[\mathrm{zero}, \mathrm{succ}]^\circ[\emptyset, 1] =$ succ$^\circ$ is inductive; $< = \mathrm{pred}^+$ with pred $=$ succ$^\circ$, which is what made §6.1's recursion terminate.

Lists: $\mathsf{mem} = [\emptyset, \pi']$ and $[\mathsf{nil}, \mathsf{cons}]^\circ[\emptyset, \pi'] = \mathsf{cons}^\circ \pi' =$ tail is inductive, so proper suffix tail^+ is; on snoc-lists init and proper prefix are.

Unique solutions

T 6.3 ✓ $S\,\mathsf{mem}(F)$ *inductive* $\Longrightarrow X = S(FX)R$ *has a unique solution* $\varphi(R,S)$*, and* $\varphi(R,S)$ *is entire if* R *and* S *are.* With $S := S^\circ$ this is the hylomorphism of §6.3, now pinned down as the *only* fixed point.

C 6.2 ✓ $g\,\mathsf{mem}(F)$ inductive $\Longrightarrow$ the unique solution of $X = g(FX)f$ is a map $(X = (\!|g^\circ|\!)^\circ (\!|f|\!)$ is entire by the theorem and simple by Ex 6.10).

C 6.3 ✓ $R^\circ\ \mathsf{mem}(F)$ inductive $\Longrightarrow (\!|R|\!)$ is surjective when R is (R surjective $\triangleq 1 \subset R^\circ R$, i.e. $\mathsf{ran}\ R = 1$).

T 6.4 ✓ R *surjective and* $Rf \subset (Ff)\,\mathsf{in} \Longrightarrow f^\circ = (\!|R|\!)$. This is the theorem §5.6 borrowed to show partition = concat$^\circ$ is a catamorphism.

Ex 6.12 ✓ R inductive $\Longleftrightarrow X = X/R$ has a unique solution.

Ex 6.17 $\mathsf{inlist} : \mathsf{list}^+\alpha \to \alpha$ is a catamorphism, but inlist on list α is not; instead $\mathsf{inlist} = \mathsf{tail}^*\ \mathsf{head}$. The same trick defines intree.

Ex 6.18 The formal definition: mem is a membership relation for F if $(1/\ \mathsf{mem})(FR) = R/\ \mathsf{mem}$ for all R; equivalently $(S/\ \mathsf{mem})(FR) = (SR)/\ \mathsf{mem}$ for all R, S.

Ex 6.20 $\mathsf{mem}(G)/\ \mathsf{mem}(F)$ is the largest lax natural transformation $F \hookrightarrow G$.

Ex 6.21 In a Boolean allegory, $\mathsf{mem}(F)$ is entire $\Longleftrightarrow F\emptyset = \emptyset$.

6.6 Sorting by selection

(6.6) ✓ $\mathsf{sort} \subset \mathsf{perm}\ (\mathsf{ordered})$, for R a *connected* preorder ($R \cup R^\circ = \Pi$). A connected preorder need not be anti-symmetric, so sort is genuinely a refinement, not an equality.

$\mathsf{ordered} = (\!|\mathsf{nil}, \mathsf{ok}\ \mathsf{cons}|\!)$, where the coreflexive ok holds at (a, x) when $(\forall b : b\,\mathsf{inlist}\,x : aRb)$ — rebuild the list, checking at each step that only R-smaller elements go in front. ($\mathsf{ok}(a,x) = (x = [] \vee aR(\mathsf{head}\ x))$ also works but is less useful here.)

Selection sort

$$
\begin{array}{ll}
 & \mathsf{perm}\ (\mathsf{ordered}) \\
= & \text{perm} = \mathsf{perm}^\circ \text{ and ordered} = \mathsf{ordered}^\circ \\
 & (\mathsf{ordered}\,\mathsf{perm})^\circ \\
= & \text{ordered} = (\!|\mathsf{nil}, \mathsf{ok}\ \mathsf{cons}|\!) \\
 & ((\!|\mathsf{nil}, \mathsf{ok}\ \mathsf{cons}|\!)\ \mathsf{perm})^\circ \\
\supseteq & \text{fusion, for a suitable select} \\
 & (\!|\mathsf{nil}, \mathsf{select}^\circ|\!)
\end{array}
$$

6.6 The fusion proviso is $\mathsf{ok}\,\mathsf{cons}\,\mathsf{perm} \supset (1 \times \mathsf{perm})\mathsf{select}^\circ$, and select is found by calculating:

$$
\begin{array}{ll}
 & \mathsf{ok}\ \mathsf{cons}\ \mathsf{perm} \\
= & \text{perm} = (\!|\mathsf{nil}, \mathsf{cons}\,\mathsf{perm}|\!)\ \ (\S 5.6) \\
 & \mathsf{ok}\ (1 \times \mathsf{perm})\ \mathsf{cons}\ \mathsf{perm} \\
= & (1 \times \mathsf{perm})\mathsf{ok} = \mathsf{ok}(1 \times \mathsf{perm}),\ \text{Ex 6.22} \\
 & (1 \times \mathsf{perm})\ \mathsf{ok}\ \mathsf{cons}\ \mathsf{perm} \\
\supseteq & \text{specifying select} \subset \mathsf{perm}\,\mathsf{cons}^\circ\ \mathsf{ok} \\
 & (1 \times \mathsf{perm})\mathsf{select}^\circ
\end{array}
$$

6.6 In words: $(a, y) = \mathsf{select}\ x$ means $[a] +\!\!+ y$ is a permutation of x with aRb for every b in y. It is undefined on [], so take $\mathsf{select} = \mathsf{embed}\ (\!|\mathsf{base}, \mathsf{step}|\!)$ with $\mathsf{embed} : \mathsf{list}\ \alpha \to \mathsf{list}^+\alpha$.

Fusion conditions: $\mathsf{base} \subset \mathsf{wrap}\,\mathsf{perm}\,\mathsf{cons}^\circ\ \mathsf{ok}$ and $(1 \times (\mathsf{cons}^\circ\ \mathsf{ok}))\ \mathsf{step} \subset \mathsf{cons}\,\mathsf{perm}\,\mathsf{cons}^\circ\ \mathsf{ok}$, satisfied by $\mathsf{base}\ a = (a, [])$ and $\mathsf{step}(a, (b, x)) = (a, [b] +\!\!+ x)$ if aRb, else $(b, [a] +\!\!+ x)$.

6.6 ✓ The hylomorphism theorem then makes $X = (\!|\mathsf{nil}, \mathsf{select}^\circ|\!)^\circ$ the unique solution of $X = (\mathsf{nil}^\circ\ \mathsf{nil}) \cup (\mathsf{select}\ (1 \times X)\ \mathsf{cons})$, i.e. $\mathsf{sort}\ x = []$ if $x = []$, else $[a] +\!\!+ \mathsf{sort}\ y$ where $(a, y) = \mathsf{select}\ x$.

Quicksort

6.6 Same shape, with a tree as the intermediate datatype: $\mathsf{tree}\ \alpha ::= \mathsf{null} \mid \mathsf{fork}(\mathsf{tree}\ \alpha, \alpha, \mathsf{tree}\ \alpha)$ and $\mathsf{flatten} = (\!|\mathsf{nil}, \mathsf{join}|\!)$, $\mathsf{join}(x, a, y) = x +\!\!+ [a] +\!\!+ y$.

$$
\begin{array}{ll}
 & \mathsf{perm}\ (\mathsf{ordered}) \\
\supseteq & \text{flatten a map, so } \mathsf{flatten}^\circ\ \mathsf{flatten} \subset 1 \\
 & \mathsf{perm}\,\mathsf{flatten}^\circ\ \mathsf{flatten}\,\mathsf{ordered} \\
= & \text{claim: } \mathsf{flatten}\,\mathsf{ordered} = \mathsf{inordered}\,\mathsf{flatten} \\
 & \mathsf{perm}\,\mathsf{flatten}^\circ\ \mathsf{inordered}\,\mathsf{flatten} \\
= & \text{converses} \\
 & (\mathsf{inordered}\,\mathsf{flatten}\,\mathsf{perm})^\circ\ \mathsf{flatten} \\
\supseteq & \text{fusion, for a suitable split} \\
 & (\!|\mathsf{nil}, \mathsf{split}^\circ|\!)^\circ\ \mathsf{flatten}
\end{array}
$$

6.6 $\mathsf{inordered} = (\!|\mathsf{null}, \mathsf{check}\,\mathsf{fork}|\!)$, with check holding at (x, a, y) when $(\forall b : b\,\mathsf{intree}\,x \Longrightarrow bRa)$ and $(\forall b : b\,\mathsf{intree}\,y \Longrightarrow aRb)$; check′ is the same with inlist for intree. Writing $Ff = f \times 1 \times f$, the proviso is $F(\mathsf{flatten}\,\mathsf{perm})\mathsf{split}^\circ \subset \mathsf{check}\,\mathsf{fork}\,\mathsf{flatten}\,\mathsf{perm}$.

Discharged by taking $\mathsf{split} \subset \mathsf{perm}\,\mathsf{join}^\circ\mathsf{check}'$, i.e. $(y, a, z) = \mathsf{split}\ x$ means $y +\!\!+ [a] +\!\!+ z$ is a permutation of x with bRa throughout y and aRb throughout z.

split = embed $(\!|\mathsf{base}, \mathsf{step}|\!)$ with $\mathsf{base} \subset \mathsf{wrap}\,\mathsf{perm}\,\mathsf{join}^\circ\mathsf{check}'$ and $(1 \times (\mathsf{join}^\circ\mathsf{check}'))\ \mathsf{step} \subset \mathsf{cons}\,\mathsf{perm}\,\mathsf{join}^\circ\mathsf{check}'$ (the book prints split for step here, and drops the $^\circ$ on join), satisfied by $\mathsf{base}\ a = ([], a, [])$ and $\mathsf{step}(a, (x, b, y)) = ([a] +\!\!+ x, b, y)$ if aRb, else $(x, b, [a] +\!\!+ y)$.

6.6 $X = (\!|\mathsf{nil}, \mathsf{split}^\circ|\!)^\circ\ \mathsf{flatten}$ is the least solution of $X = (\mathsf{nil}^\circ\ \mathsf{nil}) \cup (\mathsf{split}(X \times 1 \times X)\mathsf{join})$, i.e. $\mathsf{sort}\ x = []$ if $x = []$, else $\mathsf{sort}\ y +\!\!+ [a] +\!\!+ \mathsf{sort}\ z$ where $(y, a, z) = \mathsf{split}\ x$.

Ex 6.22 $\mathsf{inlist} = \mathsf{perm}\,\mathsf{inlist}$; hence a point-free ok and $(1 \times \mathsf{perm})\mathsf{ok} = \mathsf{ok}(1 \times \mathsf{perm})$.

Ex 6.24 $\mathsf{ordered}(R)\,\mathsf{ordered}(S) = \mathsf{ordered}(R \cap S)$.

Ex 6.25 Sorting a *set*: $\mathsf{sort} \subset \mathsf{setify}^\circ\ \mathsf{ordered}(R)$ is the wrong specification, $\mathsf{sort} \subset \mathsf{setify}^\circ\ \mathsf{ordered}(R \cap \mathsf{neq})$ the right one, where $a\,\mathsf{neq}\,b$ iff $a \neq b$.

Ex 6.30 From $\mathsf{perm} = (\!|\mathsf{nil}, \mathsf{add}|\!)$ instead, fusion gives $\mathsf{perm}(\mathsf{ordered}) = (\!|\mathsf{nil}, \mathsf{add}\,\mathsf{ordered}|\!) \supset (\!|\mathsf{nil}, \mathsf{insert}|\!)$ for a suitable map insert with $(1 \times \mathsf{ordered})\mathsf{insert} \subset \mathsf{add}\,\mathsf{ordered}$ — insertion sort.

6.7 Closure

6.7 ✓ R^*, the **REFLEXIVE TRANSITIVE CLOSURE**, is the smallest preorder containing R:

$$R \subset X \Longleftrightarrow R^* \subset X \quad \text{for every preorder } X.$$

(6.7) ✓ $R^* = (\mu X : 1 \cup (RX))$.

(6.8) ✓ $R^* = (\mu X : 1 \cup (XR))$. That $S = (\mu X : 1 \cup (RX))$ is reflexive and contains R is immediate; transitivity goes through division, the same move as Theorem 6.2.

Ex 6.32 ✓ $SR^* = (\mu X : S \cup (XR))$, $RS^* = (\mu X : R \cup (XS))$, $S^*R = (\mu X : R \cup (SX))$.

Ex 6.33 $R^* = (\!|[1,1]|\!)^\circ (\!|[1,R]|\!)$, the intermediate datatype being iterate $\alpha ::= \mathsf{once}\ \alpha \mid \mathsf{again}(\mathsf{iterate}\ \alpha)$.

Ex 6.34 $R^* = \mathsf{chainR}^\circ\ \mathsf{head}$ for a catamorphism chainR on non-empty lists.

Ex 6.36 $(R \cup S)^* = (R^* S)^* R^*$, by the diagonal rule.

Ex 6.37 $CR = C \Longrightarrow R^* = (\sim C\, R)^*$, for C coreflexive.

When the recursion has a unique solution

6.7 $X = 1 \cup (RX)$ has a unique solution — necessarily R^* — exactly when R is inductive. tail is inductive, so suffix $=$ $1 \cup (\text{tail}\,\text{suffix})$ pins suffix down, and transposing gives $\Lambda\,\text{suffix} = \langle \tau, \Lambda(\text{tail}\,\text{suffix})\rangle$ cup.

tail is undefined on [], so a case analysis splits it: $(\Lambda\,\text{suffix})[] = \{[]\}$ and $(\Lambda\,\text{suffix})([a] +\!\!+ x) = \{[a] +\!\!+ x\} \cup (\Lambda\,\text{suffix})x$ — sets as lists, and this is §5.6's tails $[] = [[]]$, $\text{tails}([a] +\!\!+ x) = [[a] +\!\!+ x] +\!\!+ \text{tails}\ x$.

Computing a closure that is not a unique fixed point

6.7 SUBTRACTION (Ex 4.30) $R - S \subset T \iff R \subset S \cup T$; hence $R - \emptyset = R$, $R \cup S = R \cup (S - R)$, $R - (S \cup T) = R - S - T$ and $(R \cup S) - T = (R - T) \cup (S - T)$.

Ex 6.35 ✓ M-CALCULUS the two defining rules are $\varphi(\mu X : \varphi X) = (\mu X : \varphi X)$ and $\varphi Y \subset Y \Longrightarrow (\mu X : \varphi X) \subset Y$. From them: *rolling* $(\mu X : \varphi(\psi X)) = \varphi(\mu X : \psi(\varphi X))$, *diagonal* $(\mu X : \mu Y : \varphi(X, Y)) = (\mu X : \varphi(X, X))$, *substitution* $(\mu X : \varphi(\mu Y : \psi(X, Y))) = (\mu X : \psi(\varphi X, X))$.

(6.9) $\theta(P, Q) \triangleq P \cup (\mu X : Q \cup ((XR) - P))$, which satisfies

$$\theta(\emptyset, S) = SR^*, \quad \theta(P, \emptyset) = P,$$

$$\theta(P, Q) = \theta(P \cup Q, (QR) - P - Q).$$

The third comes out of the rolling rule and the subtraction laws.

Read as a program: compute $\theta(\emptyset, S)$, where $\theta(P, Q) = P$ if $Q = \emptyset$, else $\theta(P \cup Q, (QR) - P - Q)$. It terminates only if SR^* is finite; then P grows at every step.

Relations as data are awkward, so apply Λ throughout: with $p = \Lambda P$, $q = \Lambda Q$, $s = \Lambda S$ and $\Lambda(SR^*) = (\Lambda S)(ER^*)$,

$$\text{close}(p, \emptyset) = p,$$

$$\text{close}(p, q) = \text{close}(p \cup q, (ER)q - p - q),$$

now with set union and set difference. This is how $E(R^*)$ is computed whenever the answer is known to be finite — reachability in a graph.

7 Optimisation Problems

7.1 Minimum and maximum

7.1 $\min R \triangleq \ni \cap (\ni^\circ \setminus R) : [\alpha] \to \alpha$ — a is a minimum of x under R when a is *in* x and is a lower bound of x. R need not be a preorder for the definition to make sense, only for it to be useful.

UNIVERSAL PROPERTY $X \subset \min R \iff X \subset \ni \wedge \ni^\circ X \subset R$.

$\max R = \min R^\circ$.

Three consequences of $f(R/S) = (f^\circ S)/R$, all about dividing by $\ni$:

(7.1) ✓ $\tau(\ni^\circ \setminus R) = R$.

(7.2) $\Lambda(S)(\ni^\circ \setminus R) = S^\circ \setminus R$.

(7.3) ✓ $\bigcup(\ni^\circ \setminus R) = \ni^\circ \setminus (\ni^\circ \setminus R)$.

(7.4) ✓ $\tau(\min R) = 1 \cap R$ — so R is reflexive exactly when the minimum of a singleton is its inhabitant.

(7.5) ✓ $\Lambda(S)(\min R) = S \cap (S^\circ \setminus R)$, whose universal property

$$X \subset \Lambda(S)(\min R) \iff X \subset S \wedge S^\circ X \subset R$$

is the workhorse of the chapter; calculations cite it as *universal property of* min.

(7.6) ✓ CONTEXT $\Lambda(S)(\min R) = \Lambda(S)(\min(R \cap S^\circ S))$ — for the purpose of minimising over the sets ΛS returns, R only has to be constrained on values related to one and the same element by S.

Fusion with the power functor

(7.7) $(ES)(\min R) = (\ni S) \cap ((\ni S)^\circ \setminus R)$, from (7.5) and $ES = \Lambda(\ni S)$.

(7.8) Shunting a map through a minimum: $(Pf)(\min R) = (\min(fRf^\circ))f$.

(7.9) R reflexive: $(PS)(\min R) = (\ni S) \cap (\ni^\circ \setminus (SR))$. One direction needs tabulations.

(7.10) ✓ FUSION WITH THE POWER FUNCTOR $(PS)(\min R) \subset (\ni S) \cap (\ni^\circ \setminus (SR))$, always — this half is all that is used later, and it follows in two lines from the naturality of $\ni$.

Distribution over union

(7.11) ✓ R a preorder: $(P(\min R))(\min R) \subset \bigcup(\min R)$ — take a minimum in each set and then a minimum of those. Only an inclusion, because a set in the collection may be empty.

(7.12) $(P(\min R))(\min R) = (P(\text{dom}(\min R)))\bigcup(\min R)$, using $\min R = (\text{dom}(\min R))(\min R)$.

Implementing min

7.1 $\min R$ refines to a map only on non-empty finite sets, and only for R a connected preorder. With setify : $\text{list}^+\alpha \to [\alpha]$ the specification is minlist $R \subset$ setify$(\min R)$, and

minlist $R = (\!|1, \text{bmin}\ R|\!)$ with $\text{bmin}\ R(a, b) = (aRb \to a, b)$ — the *leftmost* minimum, in the case of ties.

Exercises

Ex 7.1 ✓ $\text{subset}^\circ(\ni^\circ \setminus R) = \ni^\circ \setminus R$, where subset $= \ni/\ni$.

Ex 7.2 ✓ $(ER)\text{subset} = (\ni R)/\ni$.

Ex 7.4 $(PS)(\ni^\circ \setminus R) = \ni^\circ \setminus (SR)$.

Ex 7.5 ✓ R a preorder: $(\ni^\circ \setminus R)R = \ni^\circ \setminus R$.

Ex 7.6 ✓ $\min(R \cap S) = (\min R) \cap (\min S)$.

Ex 7.7 ✓ $\ni^\circ \ni = \Pi$ — every element is in some set; hence $\min R = \ni \iff R = \Pi$.

Ex 7.8 R, S reflexive: $R \cap S^\circ = (\min S)^\circ (\min R)$.

Ex 7.9 R reflexive: $R = \ni^\circ (\min R)$.

Ex 7.10 ✓ R, S reflexive: $\min R \subset \min S \iff R \subset S$.

Ex 7.11 ✓ $\min R$ is simple $\iff R$ is anti-symmetric.

Ex 7.12 R a preorder: $\min R = \ni \cap ((\min R)R)$.

Ex 7.13 R a preorder and S a map: $R \cap (SS^\circ)$ is a preorder.

Ex 7.14 ✓ R a preorder: $\Lambda(R)(\max R) = R \cap R^\circ$.

Ex 7.15 $\langle S(\min R), T(\min R)\rangle \subset \Lambda\langle S, T\rangle(\min(R \times R))$.

Ex 7.16 R reflexive, S a preorder: $\Lambda(\min S)(\min R) = \min(S; R)$, with $S; R = S \cap (S^\circ \Rightarrow R)$.

Ex 7.19 ✓ MINIMAL ELEMENTS mnl $R \triangleq \min(R^\circ \Rightarrow R)$. $(R^\circ \Rightarrow R)$ is reflexive for every R, but need not be a preorder even when R is.

Ex 7.20 ✓ $\min R \subset \text{mnl } R$, with equality exactly when R is a connected preorder.

Ex 7.22 $(Pf)(\text{mnl } R) = (\text{mnl}(fRf^\circ))f$ — Ex 7.8 for mnl.

Ex 7.23 $\text{mnl } R = \ni \iff R$ symmetric.

Ex 7.25 class $Q = \langle 1, \ni\Lambda(Q)\rangle$ cap picks an equivalence class of Q; for R a preorder, $\text{mnl}(R; S) = \Lambda(\text{mnl } R)\,\text{class}(R \cap R^\circ)\,\text{mnl } S$.

Ex 7.26 R WELL BOUNDED $\triangleq \text{dom} \ni = \text{dom}(\min R)$ — every non-empty set has a minimum. A well-bounded preorder is necessarily connected, and R is well bounded $\iff$ its strict part $R \cap \neg R^\circ$ is well-founded.

Ex 7.27 R well bounded $\Longrightarrow fRf^\circ$ well bounded, for any map f.

Ex 7.28 R well bounded $\iff \ni \subset (\min R)R^\circ$.

Ex 7.29 R a well-bounded preorder: $\bigcup(\min R) = (E(\min R))(\min R)$ — (7.11) strengthened to an equality, which is what makes setify$(\min R)$ a catamorphism.

Ex 7.30 R WELL SUPPORTED $\triangleq \text{dom} \ni = \text{dom}(\text{mnl } R)$ — weaker than well bounded.

Ex 7.31 R a well-supported preorder: $\ni \subset (\text{mnl } R)R^\circ$.

Ex 7.32 R a well-supported preorder: $\bigcup(\text{mnl } R) = (E(\text{mnl } R))(\text{mnl } R)$.

7.2 Monotonic algebras

7.2 ✓ An F-algebra $S : F\alpha \to \alpha$ is MONOTONIC ON R : $\alpha \to \alpha$ if

$$(FR)S \subset SR.$$

Addition is monotonic on $\leq$: $(\text{leq} \times \text{leq})\text{plus} \subset \text{plus leq}$, i.e. $c = a + b \wedge a \leq a' \wedge b \leq b' \Longrightarrow c \leq a' + b'$.

For a map: f monotonic on $R \iff f^\circ(FR)f \subset R \iff FR \subset fRf^\circ$; by shunting, f is monotonic on $R \iff$ on R°. Neither equivalence survives for a general S.

f DISTRIBUTES OVER R if $F(\min R)f \subset \Lambda((F\ni)f)(\min R)$ — pointwise, for $+$ over $\leq$, $\min x + \min y = \min\{a + b \mid a \in x \wedge b \in y\}$ on non-empty x, y.

T 7.1 ✓ *A map f is monotonic over $R \iff$ it distributes over R.*

Theorem (Theorem 7.2 — the greedy theorem ✓). S monotonic on the preorder R°:

$$(\!|\Lambda(S)(\min R)|\!) \subset \Lambda((\!|S|\!))(\min R).$$

Take the locally best step at every stage and the result is globally optimal. The proof is the universal property of min, then the hylomorphism theorem: the greedy fold is below every prefixed point of the associated recursion, and transitivity of R discharges it. For max the condition is monotonicity on R, not R°; and context can always be brought in, so monotonicity on $R^\circ \cap (\!|S|\!)^\circ (\!|S|\!)$ suffices.

Ex 7.33 ✓ $a + b \leq a + b' \Longrightarrow b \leq b'$, point-free.

Ex 7.34 ✓ in monotonic on $R \Longrightarrow R$ is reflexive.

Ex 7.37 ✓ A variant: f monotonic on R and $f \subset \Lambda(S)(\min R)$ $\Longrightarrow (\!|f|\!) \subset \Lambda((\!|S|\!))(\min R)$.

Ex 7.38 ✓ S monotonic on $R^\circ \Longrightarrow (\min(FR))\Lambda(S)(\min R) \subset (ES)(\min R)$ — optimal substeps.

Ex 7.39 takewhile $p = \Lambda((\text{list } p)(\text{prefix}))(\max R)$ with $R = \text{length leq length}^\circ$; with prefix $= (\!|\text{nil}, \text{cons} \cup \text{nil}|\!)$ the greedy theorem gives the standard program.

Ex 7.40 MAXIMUM SEGMENT SUM mss $= \Lambda(\text{segment sum})(\max)$, and with segment $=$ suffix prefix, mss $=$ Λ suffix $P(\Lambda(\text{prefix sum})(\max))(\max)$; the greedy theorem gives $(\!|\text{zero}, \text{oplus}|\!) \subset \Lambda(\text{prefix sum})(\max)$ with oplus $= \Lambda(\text{zero} \cup \text{plus})(\max)$, and a linear-time program follows.

Ex 7.41 filter $p = \Lambda((\text{list } p)(\text{subseq}))(\max R)$; with subseq $= (\!|\text{nil}, \text{cons} \cup \pi'|\!)$ the greedy theorem gives the standard program.

Ex 7.42 L the lexical order: $(1 \times L)\text{cons} \subset \text{cons } L$, hence $(\!|\text{nil}, \Lambda(\text{cons} \cup \pi')(\max L)|\!) \subset \Lambda(\text{subseq})(\max L)$; the greedy step is $(\text{ok} \to \text{cons}, \pi')$ with $\text{ok}(a, x)$ iff $x = []$ or $a \geq \text{head } x$.

7.3 Planning a company party

7.3 ✓ A company is a tree $\alpha ::= \text{node}(\alpha, \text{list}(\text{tree } \alpha))$; each employee has a *conviviality* rating, and no employee may attend with their immediate supervisor. Maximise the total rating of the guest list. Base functor $F(A, B) = A \times \text{list } B$.

Λ party$(\max R)$ with $R =$ (list rating sum) leq (list rating sum)$^\circ$ and party $=$ $(\!|\langle\text{include}, \text{exclude}\rangle|\!)$ choose, choose $= \pi \cup \pi'$ — one catamorphism producing the two parties, one with the root and one without.

include $= (1 \times (\text{list } \pi' \text{ concat}))\text{cons}$ (a map: take the root, so the roots of the immediate subtrees are excluded), exclude $= (1 \times (\text{list choose concat}))\pi'$ (not a map: each immediate subtree is free to include or exclude its root).

Λ party $(\max R)$
$=$ definition of party
$\Lambda((\!|\langle\text{include}, \text{exclude}\rangle|\!)\ \text{choose})(\max R)$
$=$ $\Lambda(XY) = \Lambda(X)(EY)$
$\Lambda(\!|\langle\text{include}, \text{exclude}\rangle|\!)\,(E\ \text{choose})(\max R)$
$\supseteq$ Ex 7.38, since choose is monotonic on R
$\Lambda(\!|\langle\text{include}, \text{exclude}\rangle|\!)\,(\max(R \times R))\Lambda\ \text{choose}\ (\max R)$
$\supseteq$ greedy, since $\langle\text{include}, \text{exclude}\rangle$ is monotonic on $R \times R$
$(\!|\Lambda\langle\text{include}, \text{exclude}\rangle(\max(R \times R))|\!)\ \Lambda\ \text{choose}\ (\max R)$

7.3 ✓ The two conditions are $(R \times R)\text{choose} \subset \text{choose } R$ and $(1 \times \text{list}(R \times R))\langle\text{include}, \text{exclude}\rangle \subset$

$\langle \text{include}, \text{exclude} \rangle (R \times R)$, the second reducing by Ex 7.15 to Λ include$(\max R)$ and Λ exclude$(\max R)$ separately.

include is already a map, and exclude refines to $\pi'(\text{list}(\Lambda \text{ choose}(\max R)))\text{concat}$. The result: a linear-time greedy sweep for a problem posed as an exercise in dynamic programming.

7.4 Shortest paths on a cylinder

7.4 ✓ An $n \times m$ array of positive integers rolled into a cylinder, the top and bottom rows adjacent. A path enters at the left, and from a square may step to one of the three adjacent squares of the next column. Find a path of least total cost, in $O(nm)$.

N is the type functor of n-tuples, $L = \text{list}^+$ with base functor $F(A, X) = A + (A \times X)$. The problem is Λ paths$(\min R)$ with $R = \text{sum leq sum}^\circ$ and paths : L N Nat $\to [\alpha]$.

paths is not the transpose of a catamorphism, so it is built from one: paths $= (\!|\text{generate}|\!)$ setify $\bigcup$ with generate : $F(\text{N } \alpha, \text{N P L } \alpha) \to \text{N P L } \alpha$, a *lax natural transformation*, assembled by type from generate $= F(1, \text{moves trans N} \bigcup)\,\text{zip cp N}(P \text{ in})$.

moves $x = \{\text{up } x, x, \text{down } x\}$ rotates columns, trans transposes a set of tuples, zip commutes N with F, cp $= \Lambda(F(1, \ni))$.

(7.13) ✓ The crux is Theorem 7.1, not the greedy theorem: in is a monotonic map, hence distributes, so

$$F(1, \min R)\ \text{in} \subset \text{cp}\,(P\ \text{in})\,(\min R).$$

Λ paths $(\min R)$
= definition of paths
$\Lambda(\!|\text{generate}|\!)$ setify $\bigcup(\min R)$
$\supseteq$ distribution over union (7.11), R a preorder
$\Lambda(\!|\text{generate}|\!)$ setify $(P(\min R))(\min R)$
$\supseteq$ naturality of setify
$\Lambda(\!|\text{generate}|\!)\ (\text{N}(\min R))$ setify $(\min R)$
$\supseteq$ fusion, for the Q below
$(\!|Q|\!)$ setify $(\min R)$

7.4 ✓ The fusion condition is generate $\text{N}(\min R) \supset F(1, \text{N}(\min R))Q$, and unfolding it through (7.13) and the naturalities of zip, trans and moves gives $Q =$ moves trans $(\text{N P}(\min R))(\text{N}(\min R))\ F(1, \cdot)\,\text{zip N}$ in, which as a coproduct is $Q = [\text{N wrap}, (1 \times (\text{moves trans N}(\min R)))\text{zip}'\ \text{N cons}]$.

Sets and n-tuples both become lists, $\min R$ becomes minlist R, and zip the standard function — a single left-to-right sweep, one column at a time.

7.5 The security van problem

7.5 ✓ A bank's cash must stay between 0 and N; a security van tops it up or takes a surplus. Given a sequence of transactions, cut it into as few *secure* segments as possible. The moral of the section: when the monotonicity condition fails, refine the *ordering* instead of abandoning the greedy algorithm.

ceiling $= \Lambda(\text{prefix sum})(\max \text{leq})$ and floor $= \Lambda(\text{prefix sum})(\min \text{leq})$; x is secure iff bmax(ceiling x, ceiling $x -$ floor $x) \le N$. secure is the corresponding coreflexive.

The problem is $\Lambda(\text{partition list secure})(\min R)$ with $R = \text{length leq length}^\circ$; fusing list secure into partition $= (\!|\text{nil}, \text{new} \cup \text{glue}|\!)$ gives

$$\text{partition list secure} = (\!|\text{nil}, \text{new} \cup \text{old}|\!),$$

$$\text{old} = (1 \times \text{cons}^\circ)\text{assocl}((\text{cons secure}) \times 1)\text{cons}.$$

Greedy needs [nil, new $\cup$ old] monotonic on R°, i.e.

(7.14) $(1 \times R^\circ)\text{new} \subset (\text{new} \cup \text{old})R^\circ$ — true.

(7.15) $(1 \times R^\circ)\text{old} \subset (\text{new} \cup \text{old})R^\circ$ — **false**: two partitions of the same sequence may have equal length without secure$([a] +\!\!+ x)$ implying secure$([a] +\!\!+ y)$.

secure is *prefix-closed* — secure prefix $\subset$ prefix secure — so refine R to $R; H$, where $H =$ (head prefix head$^\circ$) $\cup$ (nil$^\circ$ nil) and $R; H = R \cap (R^\circ \Rightarrow H)$: same length, and the first component a prefix.

With the refined order, (7.16) $(1 \times (R; H)^\circ)\text{new} \subset (R; H)^\circ(\text{new} \cup \text{old})$ follows from (7.18) $(1 \times \Pi)\text{new} \subset \text{new}\ H^\circ$, and (7.17) for old from the three claims (7.19)–(7.21) about the strict part $|R| = R \cap \neg R^\circ$. Hence

$$(\!|(\text{ok} \to \text{glue}, \text{new})\ (\text{wrap wrap})|\!) \subset (\!|\Lambda(S)(\min(R; H))|\!),$$

with ok holding at (a, xs) when $xs \neq []$ and $[a] +\!\!+$ head xs is secure.

secure turns out to be suffix-closed too, which is what makes the test efficient: ceiling $= (\!|\text{zero}, \text{omax plus}|\!)$ and floor $= (\!|\text{zero}, \text{omin plus}|\!)$ (two baby applications of the greedy theorem), so ok $N(a, [x] +\!\!+ xs) = (\text{omax}(a + \text{ceiling } x) - \text{omin}(a + \text{floor } x) \le N)$.

Ex 7.57 The refined order can be given directly: $Q = (\text{nil}^\circ\ !) \cup (\text{prefix}; (\text{tail}^\circ Q\ \text{tail}))$, a preorder and a linear order on partitions of one sequence. $Q \not\subset R$, yet $\Lambda((\!|S|\!))(\min Q) \subset \Lambda((\!|S|\!))(\min R)$ whenever secure is suffix-closed.